SP-4 Phase D — Multi-Provider Parallel Search SDK (detailed design)

Spec date: 2026-05-13 **Parent SP:** `docs/superpowers/specs/2026-05-12-sp4-multi-provider-redesign.md` **Prerequisite:** Phases A + B + C complete (HEAD `6db79c50`). **Status:** detailed-design locked. Implementation plan in `docs/superpowers/plans/2026-05-13-sp4-phase-d-implementation.md`.

Goal (from parent SP-4 design)

> **Phase D — Multi-provider parallel search SDK.** SharedFlow-based fan-out in `LavaTrackerSdk`. Per-provider result labeling. Search results UI updated to render labels. Cancellation + backpressure tests.

Current state audit

Already in place

Surface	What's there
`LavaTrackerSdk.multiSearch(request, providerIds, page)`	Returns `UnifiedSearchResult` after al providers complete
`LavaTrackerSdk.streamSearch(filter, providerIds, apiBaseUrl)`	SSE client connects to Lava Go API `/v search`
`UnifiedSearchResult` / `UnifiedTorrentItem` / `ProviderOccurrence` / `ProviderSearchStatus`	All four data classes exist; `DeduplicationEngine.deduplicate(.` collapses duplicates across providers
`feature/search_result/SearchResultViewModel.observeSseSearch(Filter)`	Reads SSE events from the Go API, dri `SearchResultContent.Streaming(ite activeProviders)` state

Surface	What's there
Phase C `ActiveTrackerSection`	Renders inside ProviderConfig; "Make active" mutates `LavaTrackerSdk.activeTrackerId()`

What Phase D adds

1. **Client-direct parallel fan-out for `multiSearch`** — same return shape (`UnifiedSearchResult`), but provider calls run in parallel via `coroutineScope { providerIds.map { async { ... } }.awaitAll() }`. Latency drops from `sum(latencies)` to `max(latencies)`. No API dependency.
2. **`streamMultiSearch(request, providerIds, page): Flow<MultiSearchEvent>`** — new SDK method that emits events as providers complete:
   - `MultiSearchEvent.ProviderStart(providerId, displayName)`
   - `MultiSearchEvent.ProviderResults(providerId, items)`
   - `MultiSearchEvent.ProviderFailure(providerId, reason, cause)`
   - `MultiSearchEvent.ProviderUnsupported(providerId)`
   - `MultiSearchEvent.AllProvidersDone(unified: UnifiedSearchResult)` Mirrors the SSE wire-event shape but is entirely client-side. Replaces the API dependency for users who don't want to route every search through the Lava Go service.
3. **Search-result UI consumes both paths.** `SearchResultViewModel` gets a second branch: when `filter.providerIds != null` AND no Go API endpoint is configured (or the user has opted out of API routing), use `streamMultiSearch` directly. The state shape (`SearchResultContent.Streaming(items, activeProviders)`) already supports this — only the source changes.
4. **Cancellation tests.** When the user navigates away or sets a new filter mid-search, every in-flight provider `async` MUST be cancelled. Verify by checking that the underlying `OkHttpClient.dispatcher.runningCalls()` drops to zero within 100ms of structured-concurrency cancellation.
5. **Backpressure tests.** Slow consumer (1s delay per event) MUST NOT OOM the SharedFlow buffer. Use `MutableSharedFlow(replay = 0, extraBufferCapacity = providerIds.size + 4, onBufferOverflow = BufferOverflow.SUSPEND)`. Verify by emitting 10× more events than the

buffer can hold and confirming no event is dropped (or dropped per the explicit policy if `DROP_OLDEST` is chosen for ProviderStart-style events).

6. **`ActiveTrackerSection` deletion.** Once `streamMultiSearch` is the default search path, the single-active-tracker concept is dead. Delete `ActiveTrackerSection.kt` + the `MakeActive` action + `activeTrackerId` state field. Keep `LavaTrackerSdk.activeTrackerId()` / `switchTracker()` API for transitional callers (e.g., the legacy single-provider search path) but mark them `@Deprecated("Use multiSearch/streamMultiSearch")` for removal in a later cycle.

Design decisions (locked)

D1 — SharedFlow vs coroutineScope+async

Decision: `streamMultiSearch` returns a cold `Flow<MultiSearchEvent>` backed by `channelFlow { ... }`. The internal fan-out launches one coroutine per provider via `launch { ... }` and uses `send(event)` to emit. `channelFlow` provides:

- Built-in cancellation (cancelling the collector cancels the producer coroutines)
- Built-in backpressure (`send` suspends when the consumer is slow)
- Per-emission concurrency (multiple providers can emit simultaneously without manual `Mutex`)

`MutableSharedFlow` would also work but is overkill for this use case (no multiple subscribers, no replay needs). `channelFlow` is the idiomatic choice.

For the non-streaming `multiSearch` (which returns a single `UnifiedSearchResult`), use `coroutineScope { providerIds.map { async { ... } }.awaitAll() }`. Each `async` block is independent; failures inside one don't cancel siblings (catch `Throwable` and convert to `ProviderSearchStatus.FAILURE`).

D2 — Buffer policy

Decision: `BufferOverflow.SUSPEND` for the streaming flow. Per-provider event count is bounded (1 start + 1 result + 1 failure-or-done = max 3 per provider × N providers + 1 AllDone). Even with 10 providers, the upper bound is 31 events. The default channel buffer (64) absorbs this without back-pressure mattering in practice. The chosen policy is the safe default; the backpressure test exists to prove the policy is honoured under stress (a 1s-delay consumer with 100+ emitted events MUST observe all 100 in order).

D3 — Cancellation semantics

Decision: structured concurrency via `channelFlow`. When the collector cancels (e.g., the user navigates away), the `channelFlow` block is cancelled, which cancels every child `launch` running a provider's `search`. The provider's HTTP client (Ktor/OkHttp) honours coroutine cancellation; in-flight requests are aborted at the next suspension point.

Cancellation test verifies that after `cancel()` on the collector, `OkHttpClient.dispatcher.runningCalls()` returns 0 within 200ms (allowing for OS-level connection-close latency).

D4 — Per-provider result labeling

Decision: The existing `UnifiedTorrentItem.primaryProvider` + `occurrences[].providerDisplayName` already carry the labels. The Search Results UI's existing `SearchResultContent.Streaming.activeProviders` field carries the status. No new state shape is needed. The Phase D UI work is wiring the new `streamMultiSearch` Flow into the existing state-shape — not redesigning the UI.

D5 — Default search path

Decision: when `filter.providerIds.isNullOrEmpty()` → legacy single-provider paging path (unchanged). When `filter.providerIds != null`:

- If `endpoint is Endpoint.GoApi` AND user opted in to API routing (the existing config) → SSE path (`observeSseSearch`, unchanged).
- Else → client-direct `streamMultiSearch` (NEW path, Phase D).

The two streaming paths consume the same UI state shape; the only difference is the event source (SSE wire JSON vs. Kotlin Flow events).

D6 — Phase D pre-work: `ActiveTrackerSection` removal

Decision: delete `ActiveTrackerSection.kt` + the `MakeActive` action + the `activeTrackerId` state field as PART of Phase D, not as a separate phase. The Phase C design explicitly marked this affordance as Phase D pre-work, and Phase D is the cycle that makes the single-active-tracker concept obsolete.

The Phase C addition was deliberately a small surgical affordance; deletion is symmetric.

`LavaTrackerSdk.activeTrackerId()` + `switchTracker()` API surface is RETAINED but marked `@Deprecated` — the legacy paging path still uses `activeTrackerId` to choose its tracker. Full removal of those methods is a future phase (after the legacy paging path is migrated to `multiSearch`-with-single-provider).

Affected surfaces

Files to add

- `core/tracker/client/src/main/kotlin/lava/tracker/client/MultiSearchEvent.kt` — sealed event hierarchy + `UnifiedSearchResult.from(events)` builder.
- `core/tracker/client/src/test/kotlin/lava/tracker/client/LavaTrackerSdkParallelSearchTest.kt` — unit tests verifying parallel fan-out (latencies overlap, all providers complete), cancellation (in-flight calls dropped), and backpressure (slow consumer doesn't lose events).

Files to modify

- `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`:
  - Rewrite `multiSearch(...)` body: `coroutineScope { providerIds.map { async { runOneProvider(it) } }.awaitAll() }`.
  - Add `streamMultiSearch(request, providerIds, page): Flow<MultiSearchEvent>`.
  - Mark `activeTrackerId()` + `switchTracker()` `@Deprecated`.
- `feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt`:
  - In `onCreate`, branch on `endpoint is Endpoint.GoApi` for SSE vs new `streamMultiSearch` path.
  - New `observeStreamMultiSearch(filter)` private method consuming the Flow; reuses the existing `handleSseEvent`-equivalent state-reduce logic (factor shared helpers).

Files to delete

- `feature/provider_config/src/main/kotlin/lava/provider/config/sections/ActiveTrackerSection.kt` (Phase D pre-work).
- The `ProviderConfigAction.MakeActive` data object + its `when` branch in `ProviderConfigViewModel`.
- The `activeTrackerId` state field in `ProviderConfigState`.

- The `ActiveTrackerSection(state, onAction)` invocation in `ProviderConfigScreen`.

Challenge Tests inventory

Challenge	New / changed	Scope
C32 (new) `MultiProviderParallelSearchTest.kt`	Drive: launch → Search tab → enter query → toggle on RuTracker + RuTor + ArchiveOrg → tap Search → assert all three `ProviderStreamStatus.SEARCHING` rows render → wait for results → assert UnifiedTorrentItem badges for at least 2 providers visible	Falsifiability: replace `awaitAll()` with `forEach { it.await()` (preserves sequential semantics in async wrapper) Test SHOULD still pass eventually (no provider is dropped) but the test asserts max-total-time < 2× single-provider-latency — sequential exec would fail that bound.
C33 (new) `SearchCancellationTest.kt`	Drive: start a multi-provider search → press back before any results arrive → assert no in-flight `OkHttpClient` calls remain after 200ms	Falsifiability: remove the `coroutineScop` wrapper so `async`'s are unstructured (won't be cancelled by parent cancellation). Test fails — calls keep running.

Both are gating-emulator-bound (operator runs them; KDoc carries the rehearsal protocol).

Existing Challenges affected:

- **C04** (just rewritten in Phase C) — no change needed; the "Sync this provider" assertion still holds because `SyncSection` survives Phase D.
- **C01** — no change needed; the menu structure is unchanged.

Anti-bluff posture

Per §6.J / §6.L:

1. The new SDK methods MUST have unit tests against real `TrackerRegistry` + fake providers, not mocked SDK behavior.
2. Cancellation test MUST observe a real `OkHttpClient.dispatcher.runningCalls()` count, not a mock.
3. Backpressure test MUST send N+1 events through the Flow with a deliberately-slow collector and assert no event is silently dropped.
4. Bluff-Audit stamps for every new `*Test.kt`; the C32/C33 rehearsals are documented in their KDoc and executed by the operator on the gating emulator before next tag.
5. `@Deprecated` markers on `activeTrackerId()` + `switchTracker()` MUST cite the migration target (`multiSearch` / `streamMultiSearch`).
6. §6.S CONTINUATION updated in the implementation commit.

Out of scope for Phase D

- **Per-provider rate limiting** — A provider serving slow responses (Cloudflare-mitigated rutracker) should not slow the rest. Handled implicitly by `awaitAll()` returning when the slowest completes; explicit per-provider timeouts are a Phase D+1 polish item if observed.
- **Result-ranking weights per provider** — `DeduplicationEngine` currently uses occurrence count; weighted scoring is a future SP.
- **Removal of the SSE path** — both paths coexist post-Phase-D. SSE remains the default when the Go API is configured (server-side dedup, server-side cache); client-direct is the fallback when API is offline.
- **Removing `LavaTrackerSdk.activeTrackerId()` / `switchTracker()`** — only `@Deprecated`-marked here. Full removal awaits the legacy paging path migration.

Operator's invariants reasserted

- §6.J anti-bluff: cancellation + backpressure tests assert on observable state (running-calls count, event sequence), not on mock-call counts.

- §6.Q Compose layout: no UI changes risk nested-scroll antipatterns (the SSE-vs-streamMultiSearch branch is in the ViewModel; the Compose tree is unchanged).
- §6.R no hardcoded literals: no new literals (timeouts come from existing config; provider IDs come from `LavaTrackerSdk.listAvailableTrackers()`).
- §6.S CONTINUATION updated in the implementation commit.
- §6.W mirrors: pushed to both in lockstep.

Implementation plan

See `docs/superpowers/plans/2026-05-13-sp4-phase-d-implementation.md` — 6 tasks, ~20 steps.

Forensic anchor

The sequential `multiSearch` was acceptable for v1.2.x with 2 providers. With Phase A+B making it ergonomic to add custom providers (clone-with-new-name + per-provider config), the sequential cost compounds. A user with 6 providers and 2-second average latencies saw 12-second total search time; parallel fan-out brings that to ~2.5 seconds. That's the user-visible win. The cancellation + backpressure tests are the constitutional load-bearing guarantees that the parallel path doesn't regress reliability.